# Product Requirement Document (PRD)

**Project Name:** Autonomous E-Commerce Operations & Inventory System
**Target Deployment:** Kaggle AI Agents Capstone (June 2026 Cohort) & Internal Simulation Suite
**Data Strategy:** Local Synthetic/Mock Data (Self-Contained MVP)
**Date:** June 21, 2026
**Status:** Approved / Baseline Updated

## 1. Executive Summary

This project outlines a self-contained multi-agent automation system built using the Google Agent Development Kit (ADK). The system is engineered to simulate and automate premium e-commerce catalog workflows. By orchestrating three distinct AI agents running over a local Model Context Protocol (MCP) server, the software autonomously audits mock inventory pools, evaluates margin health based on generated synthetic market rates, and drafts sophisticated product copywriting.

## 2. Problem Statement & Objectives

Managing a premium retail catalog requires high precision. Testing multi-agent systems against live production databases and commercial external APIs during an MVP phase introducing unpredictable billing, rate-limiting constraints, and external dependencies that hinder developer velocity and demonstration stability.

**Objective:** Build a fully local, deterministic ADK-orchestrated pipeline that demonstrates margin analysis, structural catalog sanitization, and brand-aligned metadata generation using a secure, mock-data framework.

## 3. Scope & Agent Workflows (The "In-Scope" Solution)

The system utilizes an ADK supervisor-worker topology consisting of three primary agents interacting with a local simulation layer:

- **Agent A: The Market Monitor (Data Ingestion Simulation)**
  *Trigger:* Sequential system execution call.
  *Action:* Connects to a local MCP server that encapsulates a synthetic data provider.
  *Logic:* Fetches the mock spot price of platinum generated from a local configuration file (`mock_pricing_seed.json`). It calculates simulated gross margin metrics against mock SKU

profiles. If a simulated price drop creates an artificial margin violation, it flags the item in the shared session state.

- **Agent B: The Catalog Auditor (Logic & Rules Engine)**
  *Trigger:* Receives the data payload from Agent A.
  *Action:* Scans a local test database (`mock_inventory.sqlite`) populated with diverse synthetic SKU configurations.
  *Logic:* Enforces structural guardrails. It programmatically validates catalog composition, executing the core operational rule that **two-tone jewelry pieces must be treated as completely distinct SKUs from pure platinum items**. If a mock two-tone profile violates this structure, Agent B logs a descriptive validation block.
- **Agent C: The Brand Voice Copywriter (Execution Simulation)**
  *Trigger:* Receives validated, distinct SKU schemas passed forward by Agent B.
  *Action:* Generates final front-end product descriptions and metadata strings.
  *Logic:* Adheres to a strict stylistic boundary prompt. The generated copy must bypass aggressive promotional language, focusing entirely on **simplicity and sophistication** to align with a luxury brand identity.

## 4. Out of Scope (V1 / Capstone Limits)

To guarantee execution before the July 6th deadline, the following functionalities are strictly excluded:

- **Live Ingress/Egress web traffic:** No external HTTP connections to active live e-commerce vendor portals or real-time web pricing engines.
- **Write Access to Live Infrastructure:** All persistence operations occur purely inside local runtime variables or isolated mock SQLite files.
- **Public Web Interfaces:** The presentation layer is confined to a single-node developer console dashboard.

## 5. Technical Requirements & ADK Optimization

- **Framework:** Google Agent Development Kit (ADK) configured for sequential execution pipelines and managed local state passing.
- **Model Configuration:**
  - *Gemini 2.0 Flash:* Orchestrated for Agent A and Agent B to process quick parameter extraction and rule checks.
  - *Gemini 1.5 Pro:* Orchestrated for Agent C to maintain deep contextual memory for premium text synthesis.

- **Data Transport:** Model Context Protocol (MCP) client-server loop running entirely over local standard input/output channels (`stdio`).

## 6. Success Metrics & Acceptance Criteria

### Engineering & Capstone Criteria

- **Zero External Dependencies:** The system executes completely offline or without external API keys (outside of the Google Gemini API framework).
- **Deterministic Handoffs:** Structured JSON states pass cleanly from Agent A to Agent B to Agent C within the ADK environment.
- **Simulation Fidelity:** The mock data layer reliably surfaces edge cases (such as artificially high material spikes or mislabeled mock SKUs) to demonstrate the agents' evaluation logic during the final presentation.

### Business Logic Validation

- **SKU Separation verification:** When processing the mock dataset, Agent B catches 100% of the embedded "two-tone" catalog errors and issues a clean validation error block.
- **Voice Alignment verification:** Agent C generates mock catalog listings that strictly conform to a minimalist style guide, utilizing zero promotional buzzwords.